OBJECT-ORIENTED PROGRAMMING LAB — PROJECT PROPOSAL

DictatorScript

A Custom Transpiling Programming Language

Inspired by the Language of Absolute Command

Instructor
Muhammad Khalid Khan

Team Members
Taher Mustansir — 25K-0119
Noman Kamran — 25K-0110
Manaam Siddique — 25K-0135

FAST NUCES, Karachi

Submission Date: 27 February 2026

1. Project Overview

DictatorScript is a custom programming language designed and implemented as an Object-Oriented Programming lab project. The core deliverable is a working transpiling compiler — the dsc binary — that reads .ds source files, processes them through a full compiler pipeline, and outputs valid C++ code which is then compiled by g++ into a native executable.

The language is thematically flavored around authoritarian command structures, inspired by the rhetorical style of historical dictatorships. This gives every keyword a personality: variables are declared, functions issue commands, loops impose their will, and conditionals interrogate conditions. The theme is intentionally bold and memorable, making it stand out as a creative and technically serious project.

Importantly, this is not just a themed toy. DictatorScript supports real data types, structs, pointers, dynamic memory, arrays, hashmaps, multi-file imports, and a full VS Code syntax highlighting extension — making it a genuine, usable language.

> **GitHub Repository**
> `https://github.com/TaherMustansir1929/DictatorScript`

2. Project Objectives

This project is designed to demonstrate practical mastery of OOP concepts through a real-world software system. The specific goals are:

- **Design a Complete Language:** Define the syntax, grammar, keywords, data types, operators, and control flow of DictatorScript from scratch.
- **Build a Multi-Stage Compiler:** Implement each compiler stage — Lexer, Parser, Semantic Analyzer, Code Generator — as a separate, well-encapsulated C++ class.
- **Apply Core OOP Principles:** Demonstrate encapsulation, abstraction, inheritance, and polymorphism through the compiler's class hierarchy and the Visitor design pattern used in the AST stages.
- **Create a Developer Tool:** Build a VS Code extension (dictatorscript-vscode) using TypeScript that provides syntax highlighting for .ds files.
- **Produce Working Programs:** Compile real DictatorScript programs — hello world, arithmetic, loops, functions, structs — into runnable native executables.

3. Language Design

3.1 Theme and Concept

DictatorScript maps every C++ construct to a thematic keyword drawn from the vocabulary of authoritarian rule. The result is a language that is immediately recognizable, memorable, and fun to write in, while remaining a technically accurate abstraction over C++.

3.2 Keyword Reference Table

DictatorScript Syntax	C++ Equivalent	Thematic Meaning
`regime start { }`	int main() { }	Entry point — seat of power
`declare int x = 5`	int x = 5;	Variable declaration
`declare elite int x`	const int x	Constant — elite status
`command add(...) -> int`	int add(...)	Function definition
`report value`	return value;	Return from function
`interrogate (cond)`	if (condition)	Conditional check
`otherwise interrogate`	else if	Secondary condition
`otherwise { }`	else { }	Fallback branch
`impose (i from [1..5])`	for (int i=1; i<=5; i++)	Range-based for loop
`impose (condition)`	while (condition)	While loop
`impose forever`	while (true)	Infinite loop
`silence`	break;	Break out of loop
`proceed`	continue;	Skip iteration
`broadcast(...)`	std::cout << ...	Print output
`demand(x)`	std::cin >> x	Read user input
`loyal / traitor`	true / false	Boolean literals
`law Point { int x; }`	struct Point { int x; }	Struct definition
`summon int`	new int	Heap allocation
`kill ptr`	delete ptr	Memory deallocation
`import "utils"`	#include "utils.ds"	File import

3.3 Data Types

DictatorScript Type	C++ Equivalent
int	int
float	float
double	double

char	char
string	std::string
bool	bool
void	void
int[]	std::vector<int>
int->	int* (raw pointer)
map<K, V>	std::unordered_map<K, V>

3.4 Sample Programs

The following program demonstrates entry point, variable declaration, user input, and conditional logic:

```
// DictatorScript — Conditional Example

regime start {
    declare int age
    broadcast("Enter your age: ")
    demand(age)

    interrogate (age >= 18) {
        broadcast("You are a loyal citizen.\n")
    } otherwise interrogate (age >= 13) {
        broadcast("You are a young recruit.\n")
    } otherwise {
        broadcast("Too young for the regime.\n")
    }
}
```

This example shows a function, a range-based loop, and returning a value:

```
// DictatorScript — Function with Loop

command sumSquares(int n) -> int {
    declare int total = 0
    impose (int i from [1..n]) {
        total = total + (i * i)
    }
    report total
}

regime start {
    declare int result = sumSquares(5)
    broadcast("Sum of squares 1..5 = " + result + "\n")
}
```

This example shows pointers, structs, and dynamic memory:

```
// DictatorScript — Structs & Pointers

law Point {
    int x
    int y
}

regime start {
    declare Point-> p = summon Point
    p->x = 10
    p->y = 20
    broadcast("Point: " + p->x + ", " + p->y + "\n")
    kill p
}
```

4. Compiler Architecture

The dsc compiler processes .ds source files through a five-stage pipeline. Each stage is implemented as a separate C++ class, communicating with adjacent stages through clean interfaces. This is a direct application of the encapsulation and separation-of-concerns principles from OOP.

```
  .ds Source File
       |
  [ Lexer ]          Tokenizes raw text into a stream of typed Tokens
       |
  [ Parser ]         Recursive descent; builds an Abstract Syntax Tree
       |
  [ SemanticAnalyzer ] Type checking, scope validation (Visitor Pattern)
       |
  [ CodeGenerator ]  Emits valid C++ source code (Visitor Pattern)
       |
  [ .cpp file ]
       |
  [ g++ ]            Compiles to a native executable
       |
  [ Executable ]
```

4.1 Stage Breakdown

Stage 1 — Lexer

Reads the .ds source file character by character and produces a flat list of Token objects. Each Token stores its type (keyword, identifier, literal, operator, punctuation), raw text, and source line number for accurate error messages. The Lexer handles both single-line (//) and multi-line (/* */) comments.

Stage 2 — Parser

Takes the token stream and builds an Abstract Syntax Tree (AST) using recursive descent parsing — one method per grammar rule. The AST is a tree of ASTNode-derived objects, where each node represents a language construct: declarations, expressions, conditionals, loops, function calls, and so on.

Stage 3 — Semantic Analyzer

Traverses the AST using the Visitor Pattern and performs semantic validation: checking that variables are declared before use, detecting type mismatches, catching duplicate declarations, and validating function signatures. Errors are collected by the ErrorReporter and reported after analysis with precise line numbers.

Stage 4 — Code Generator

Also uses the Visitor Pattern to walk the validated AST and emit the corresponding C++ source code. Each ASTNode type has a corresponding visit() method in the CodeGenerator that outputs correct, compilable C++. The result is a .cpp file that compiles cleanly with g++.

Stage 5 — g++ Invocation

The Compiler orchestrator class manages the full pipeline and, after code generation, invokes g++ via a subprocess call to compile the emitted .cpp into a native binary. From the user's perspective, they run dsc on a .ds file and get an executable.

4.2 Compiler CLI Reference

```
dsc source.ds                   # Compile .ds to executable
dsc source.ds -o myprogram      # Compile with custom output name
dsc source.ds --emit-cpp        # Emit .cpp only, skip g++
dsc source.ds --dump-tokens     # Debug: print token stream
dsc source.ds --dump-ast        # Debug: print AST structure
dsc source.ds --verbose         # Print each pipeline stage
```

5. OOP Concepts Demonstrated

DictatorScript is an ideal showcase for OOP because building a compiler naturally requires every major OOP principle. The table below maps each concept to a concrete implementation:

OOP Concept	How It Appears in DictatorScript
Encapsulation	Each stage (Lexer, Parser, SemanticAnalyzer, CodeGenerator) is a class that hides its internals. Only the required interface is exposed (e.g., tokenize(), parse(), analyze(), generate()).
Abstraction	ASTNode is an abstract base class. The compiler works with the abstract tree interface without needing to know which specific node type it is handling.
Inheritance	All AST node classes inherit from ASTNode. Specialized types like BinaryExprNode, IfNode, ForNode, FunctionNode extend base classes and add their own fields.
Polymorphism	The Visitor Pattern uses polymorphic dispatch: each ASTNode calls accept(visitor), which dispatches to the correct visit() method in the SemanticAnalyzer or CodeGenerator.
Visitor Pattern	A formal design pattern that separates the AST structure from the operations on it. Adding a new pass (e.g., an optimizer) only requires a new Visitor class — no changes to the AST.
Composition	The Compiler class is composed of a Lexer, Parser, SemanticAnalyzer, and CodeGenerator — a clean example of has-a relationships over is-a.
Single Responsibility	Each class has one job. The Lexer tokenizes. The Parser parses. The SemanticAnalyzer validates. The CodeGenerator emits. No class does more than one thing.
Error Handling	The ErrorReporter is a shared object passed through all stages. It accumulates errors with line numbers and reports them cleanly at the end — a reusable cross-cutting service.

6. Project File Structure

```
DictatorScript/
├── CMakeLists.txt                  Build system (CMake 3.16+)
├── README.md                       Full language & compiler documentation
├── sample-code.txt                 Example .ds programs
│
├── src/
│   ├── main.cpp                    dsc entry point & CLI argument handling
│   └── compiler/
│       ├── Compiler.h / .cpp       Pipeline orchestrator
│       ├── ErrorReporter.h / .cpp  Error accumulation & formatted reporting
│       ├── Token.h / .cpp          Token data structure
│       ├── TokenType.h             Enum of all token types
│       ├── Lexer.h / .cpp          Stage 1: Tokenizer
│       ├── ASTNode.h / .cpp        Abstract base class & all node subtypes
```

```
│       ├── Parser.h / .cpp           Stage 2: Recursive descent parser
│       ├── SymbolTable.h / .cpp      Scoped symbol table for variables/functions
│       ├── SemanticAnalyzer.h / .cpp Stage 3: Type checking & scope validation
│       └── CodeGenerator.h / .cpp   Stage 4: C++ code emission
│
├── dictatorscript-vscode/            VS Code extension (TypeScript)
│   └── syntaxes/                    TextMate grammar for .ds syntax highlighting
│
└── examples/
    ├── hello_world.ds
    ├── sample.ds
    └── multifile/                    Multi-file import demo
```

7. Development Plan & Timeline

The project is split into four phases timed across the semester. Workload is distributed to match the team's current skill level while ensuring all deliverables are achievable.

Phase	Task	Deliverable	Timeline
1	Language Design & Lexer	Complete keyword set defined. Lexer class implemented and tested on sample .ds files.	Weeks 1 – 2
2	Parser & AST	Recursive descent Parser implemented. Full ASTNode class hierarchy built. Tree printed for test programs.	Weeks 3 – 4
3	Semantic Analyzer & Code Generator	Visitor Pattern implemented. Type checking works. C++ code emitted and compiled by g++.	Weeks 5 – 6
4	VS Code Extension & Final Testing	dictatorscript-vscode extension published. All example programs compile and run correctly.	Week 7

8. Team Roles & Responsibilities

Student	Roll No.	Primary Responsibility
Taher Mustansir	25K-0119	Compiler Architecture, Parser, Code Generator, Project Lead & Integration
Noman Kamran	25K-0110	Lexer, Token Design, Semantic Analyzer, Error Reporting System

Manaam Siddique	25K-0135	AST Node Design, VS Code Extension, Testing & Example Programs

9. Tools & Technologies

- **Primary Language:** C++17 — entire compiler is written in C++
- **Build System:** CMake 3.16+ — cross-platform build configuration
- **C++ Compiler:** g++ / clang++ / MSVC — builds dsc and compiles emitted C++ code
- **Extension Language:** TypeScript — used to write the VS Code syntax highlighting extension
- **Version Control:** Git + GitHub (github.com/TaherMustansir1929/DictatorScript)
- **IDE:** Visual Studio Code with the custom dictatorscript-vscode extension

10. Conclusion

DictatorScript is not a typical OOP assignment. Rather than building an isolated application with a handful of classes, this project constructs a complete software system — a real, working programming language — from the ground up. Every component maps directly to the OOP concepts covered in this course.

The Lexer, Parser, SemanticAnalyzer, and CodeGenerator each demonstrate encapsulation and single responsibility. The ASTNode class hierarchy demonstrates inheritance and abstraction. The Visitor Pattern in the analyzer and generator stages is a textbook example of polymorphism. The Compiler class demonstrates composition. The ErrorReporter demonstrates a reusable cross-cutting service.

The thematic dictator framing makes the language memorable and gives the project a clear identity. But the substance underneath is serious: a working multi-stage compiler, a VS Code extension, and a language that compiles real programs. By the end of this project, all three team members will have practical experience with how programming languages are designed, how compilers work, and how large C++ systems are structured using proper OOP design.

We believe DictatorScript is appropriately scoped for second-semester students — challenging enough to be educational, focused enough to be completable — and we look forward to delivering a working compiler by the end of the semester.

"All code must obey. The regime has spoken."
— DictatorScript Manifesto